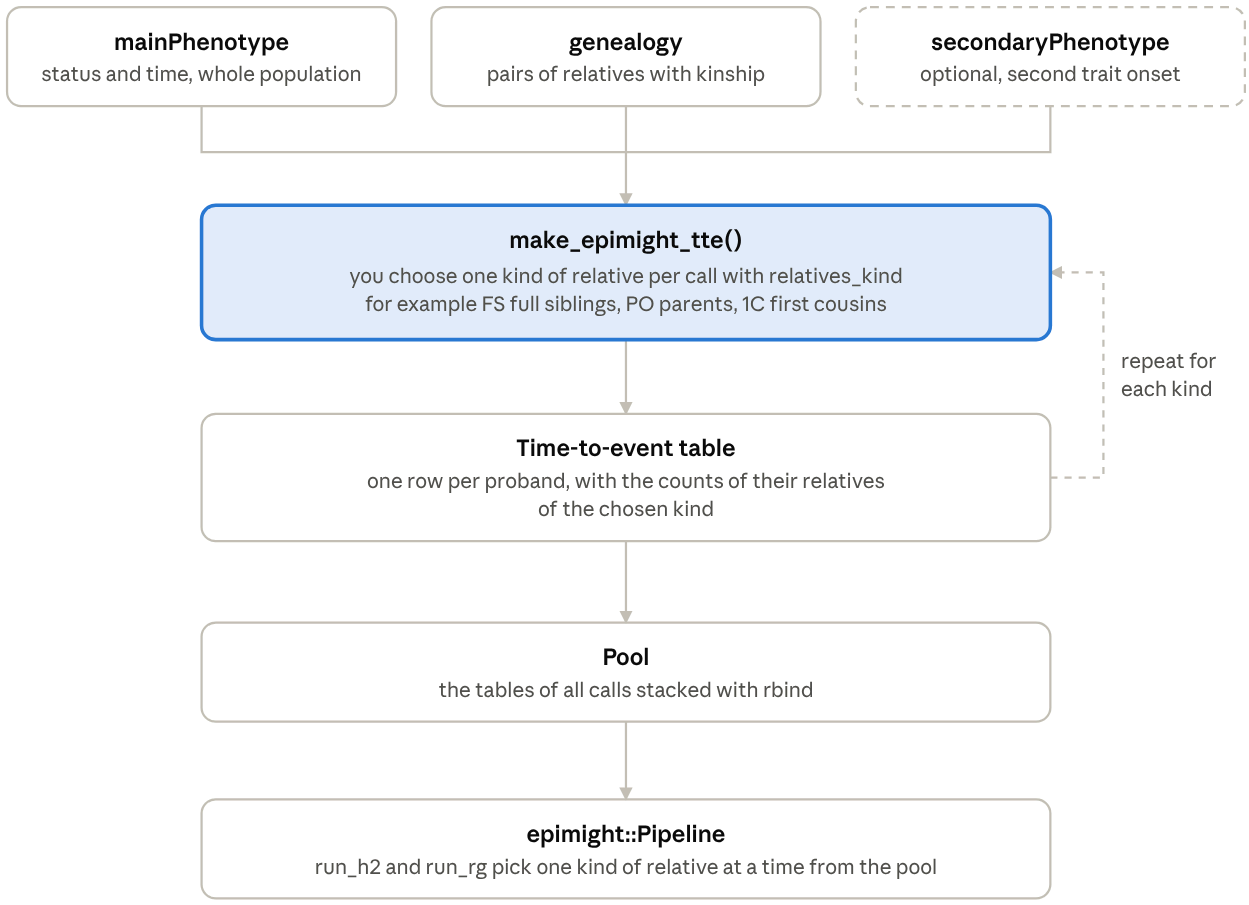
make_epimight_tte: user manual

Oct 6, 2026 · @Someone

1. About this manual

This manual takes you from your own register tables to a finished epimight run, one step at a time. `make_epimight_tte()` builds the time-to-event table that epimight needs, for one trait and one kind of relative per call.

One call returns the probands with counts for the kind of relative you choose



pipeline · 3 tables in, 1 pool out

Every call returns the same probands. What differs between calls is whose diagnoses are counted next to each proband. One call can count the proband's full siblings, the next their parents, the next their first cousins. There is no separate table of siblings or of parents.

The output has one row per proband. Each row holds the proband's own outcome (status and age) and two counts. `relatives_n` is the number of relatives of that kind with phenotype data. `relatives_n_trait` is the number of those who are affected.

Sections 3 to 9 are the steps, in the order you do them. Sections 10 to 13 are reference material for when something looks wrong.

2. Before you start

You need three things in your R session.

- [ ] R with the `data.table` package installed. The function loads it itself.
- [ ] The `epimight` package, for the last step.
- [ ] The file `make_epimight_tte.R`, loaded with `source()`.

```
source("make_epimight_tte.R")
```

You also need two tables in memory, and a third one for some analyses.

Table	Needed	Holds
`mainPhenotype`	always	Status and time for the index trait, for everyone in the population
`genealogy`	always	Pairs of relatives with their relationship code and kinship
`secondaryPhenotype`	only for second-trait exclusion	Onset of the second trait

The function never writes to disk and does not change your input tables. It prints a log line for every filter, so keep the console output of each run.

3. Step 1. Prepare the input tables

Build the tables with exactly these column names. Names are case sensitive. A missing column stops the run with a message that names it. Extra columns are ignored.

mainPhenotype

Long format, one row per person per phenotype. This table supplies the probands and the status of every relative. It must therefore cover the whole population, unaffected people included. A person without a row for the trait is never counted as a relative.

Column	Type	Content	Rule enforced by the code
`ID`	any, stored as character	Person identifier, same key as in the genealogy	Duplicate IDs within one trait stop the run
`Sex`	any, stored as character	Sex in your own coding	Passed through to the output as `sex`
`BirthDate`	Date or text such as 1985-03-14	Date of birth	Must parse as a date, otherwise the run stops
`Phenotype`	character	Phenotype name	Must match the `trait` argument exactly
`Status`	integer 0, 1 or 2	0 censored, 1 affected, 2 competing risk such as death	Any other value, including missing, stops the run
`FailureTime`	numeric, years	Years from birth to diagnosis, censoring or competing event. Decimals allowed	Rows that are missing or negative are dropped and counted in the log

People dropped for a missing or negative `FailureTime` disappear as probands and as relatives.

```
ID,Sex,BirthDate,Phenotype,Status,FailureTime
1001,F,1980-05-02,MDD,1,24.6
1002,M,1982-11-19,MDD,0,39.1
1003,F,1955-01-30,MDD,2,61.3
1004,M,1953-07-08,MDD,0,68.5
```

The example holds one disorder, with one row per person. If your file holds several disorders, each person has one extra row per disorder and the function reads only the rows of the `trait` you ask for.

genealogy

One row per pair of relatives. Pairs may be stored once or in both directions. Duplicates and self pairs are removed by the function.

Column	Type	Content
`id1`	any, stored as character	First person of the pair
`id2`	any, stored as character	Second person of the pair
`relationship_kind`	character	Relationship code used in your file
`kinship`	numeric	Kinship coefficient of the pair. The function returns 2 x kinship as relatedness

```
id1,id2,relationship_kind,kinship
1001,1002,FS,0.25
1001,1003,PO,0.25
1001,1004,PO,0.25
1002,1003,PO,0.25
```

The codes below are the ones used in the function's examples. The kinship values are the standard expectations, so check them against your own file.

Code	Relationship	Expected kinship	Relatedness passed to run_h2
`FS`	Full siblings	0.25	0.5
`PO`	Parent and offspring	0.25	0.5
`mHS`	Maternal half siblings	0.125	0.25
`pHS`	Paternal half siblings	0.125	0.25
`Av`	Avuncular	0.125	0.25
`1C`	First cousins	0.0625	0.125

secondaryPhenotype

Same long format, used only to find when a proband got the second trait. Only rows with Status 1 and a non-missing `FailureTime` are read. If a person has several such rows, the earliest onset is used. If both traits sit in one long file, pass that file here as well.

Column	Type	Content
`ID`	any, stored as character	Person identifier
`Phenotype`	character	Must match the `trait2` argument exactly
`Status`	integer	Only 1 is used
`FailureTime`	numeric, years	Age at onset of the second trait

4. Step 2. Choose the relatives

Each call handles one kind of relative. Pick the row that matches the kind you want and use its settings. The labels are free text, but run_h2 later selects on the label, so it must describe the relationship.

Relatives wanted	`relatives_kind`	Suggested `relatives_label`	`relative_is`	`vertical`	`max_relatives`
Full siblings	`"FS"`	`"full_siblings"`	`"either"`	`"none"`	not needed
Maternal half siblings	`"mHS"`	`"maternal_half_siblings"`	`"either"`	`"none"`	not needed
Paternal half siblings	`"pHS"`	`"paternal_half_siblings"`	`"either"`	`"none"`	not needed
All half siblings pooled	`c("mHS", "pHS")`	`"half_siblings"`	`"either"`	`"none"`	not needed
Parents	`"PO"`	`"parents"`	`"either"`, or the parent column if your file is directional	`"older"`	2
Children	`"PO"`	`"children"`	`"either"`, or the child column if your file is directional	`"younger"`	not needed
First cousins	`"1C"`	`"first_cousins"`	`"either"`	`"none"`	not needed
Avuncular	`"Av"`	`"avuncular"`	`"either"`	`"none"`	not needed

Two rules for this step. Pool several codes under one label only when they share the same relatedness. Use `vertical = "none"` for avuncular pairs, because `vertical` compares birth order and an uncle can be younger than a niece.

How a relative gets counted

A relative adds to a proband's counts only after passing five gates in this order.

1. **Phenotype data.** The relative has a row for the trait in `mainPhenotype`. Relatives without data are dropped silently.
2. **Observation window.** With `max_followup_relatives` set, the relative was born early enough to be observed for the full window. Step 3 explains this.
3. **Code.** The pair's `relationship_kind` is one of the codes in `relatives_kind`.
4. **Direction.** `relative_is` decides which column holds the relative. With `"either"` every pair is read both ways.
5. **Birth order.** With `vertical = "older"` only relatives born before the proband are kept. With `"younger"` only those born after. A relative born on the same day is never kept under either setting.

The relatives left are summed per proband. Only Status 1 makes a relative affected. A Status 2 relative adds to `relatives_n` but not to `relatives_n_trait`. Without a window for relatives, a relative's diagnosis counts at any age, also after the proband's own follow-up ended.

Worked example

Five people have phenotype data for MDD. A sixth sibling, F, is in the genealogy but has no phenotype row. No observation window is set.

Person	Born	Status	Role in the family
A	1980	1 affected	Child
B	1982	0 censored	Child
C	1985	2 competing risk	Child
D	1955	1 affected	Mother of A, B and C
E	1953	0 censored	Father of A, B and C
F	1987	no row	Child, missing from the phenotype file

The genealogy holds every sibling pair among A, B, C and F as `FS`, and each child paired with D and with E as `PO`.

Proband	FS, vertical none	PO, either, vertical none	PO, either, vertical older
A	n 2, affected 0	n 2, affected 1	n 2, affected 1
B	n 2, affected 1	n 2, affected 1	n 2, affected 1
C	n 2, affected 1	n 2, affected 1	n 2, affected 1
D	n 0, affected 0	n 3, affected 1	n 0, affected 0
E	n 0, affected 0	n 3, affected 1	n 0, affected 0

F is never counted, because F has no phenotype row. C has Status 2, so C adds to the siblings' `relatives_n` but not to their affected count. The middle column shows the mistake to avoid. `PO` read both ways without `vertical` turns the children into "parents" of D and E.

Probands with no relatives of the kind stay in the output with both counts at 0. Do not remove them, because epimight's population CIF uses every row of the trait in the pool.

5. Step 3. Set the observation time

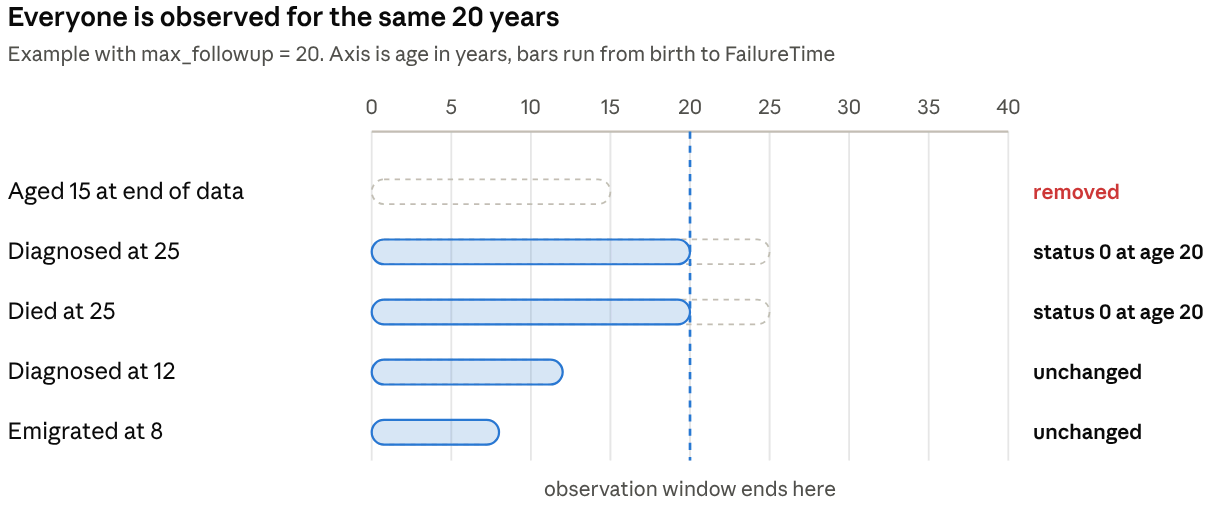
Four arguments control how long people are observed. Pick by what you want to achieve.

Goal	Argument	Applies to
Everyone observed for exactly the same number of years	`max_followup`	Probands
The same, for relatives, with its own number of years	`max_followup_relatives`	Relatives
Only drop people born too recently, keep all later follow-up	`min_followup`	Probands
Tell the function when your data end	`study_end`	All three arguments above

Always set `study_end` to your administrative end date, for example `study_end = "2016-12-31"`. If you leave it empty, the function takes the latest birth date plus `FailureTime` among Status 0 people for this trait and prints the date. Check that printed date.

A fixed observation window (max_followup)

Use this to compare birth cohorts on equal terms. With `max_followup = 20`, a person born in 1960 and a person born in 1980 are both observed for exactly 20 years from birth.



fixed observation window · 5 example people

Situation	Result
Born less than 20 years before the end of data	Removed
Diagnosis, death or censoring after age 20	Kept, `FailureTime` set to 20 and Status set to 0
Diagnosis, death or emigration at or before age 20	Unchanged

The filled part of each bar is what the output keeps. The dashed outline is what the input held. An event at exactly the window age is kept as an event.

A separate window for relatives (max_followup_relatives)

Relatives follow the same three rules with their own number of years. A relative diagnosed after their window still counts in `relatives_n` but not as affected. A relative born too late to be observed for their full window is not counted at all.

You set	Window for probands	Window for relatives
`max_followup = 20`	20 years	20 years, the default copies the proband value
`max_followup = 20`, `max_followup_relatives = 50`	20 years	50 years
`max_followup = 20`, `max_followup_relatives = NULL`	20 years	none, counted at any age
`max_followup_relatives = 50` only	none	50 years

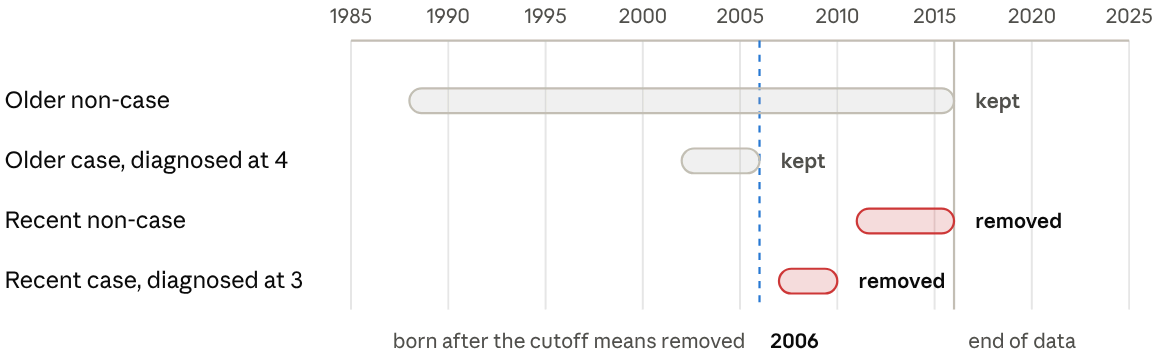
Use a longer window for relatives who are older than the probands, such as parents. Keep `filter_relatives = FALSE` when the two windows differ. With `TRUE` the relatives are taken from the already filtered probands, so the shorter of the two windows wins.

A minimum follow-up only (min_followup)

Use this when you only want to drop people born too recently and keep everything that happens later. `min_followup = 10` removes probands born less than 10 years before the end of data, cases and non-cases alike. Nothing is censored.

Birth date decides who stays, not age at diagnosis

Example with min_followup = 10 and data ending in 2016. Bars run from birth to end of follow-up



minimum follow-up filter · 4 example probands

The filter looks at birth date alone, so an early diagnosis in someone born before the cutoff is kept. It applies to probands only, unless `filter_relatives = TRUE`. You do not need it together with `max_followup`, which already removes people born too late. If you set both, both are applied.

What the window does not fix

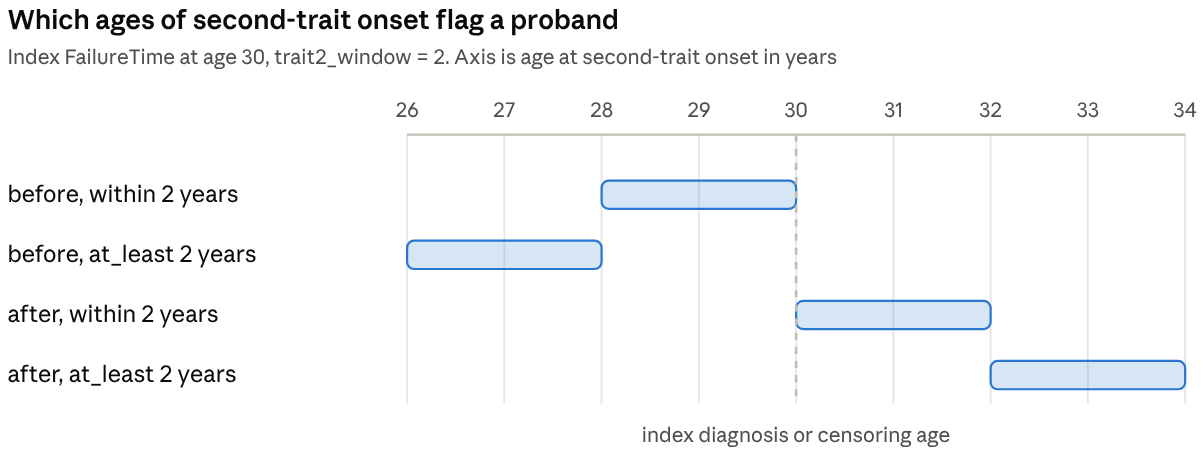
A fixed window equalises how long people are observed, not when. Someone born in 1960 is observed during 1960 to 1980 and someone born in 1980 during 1980 to 2000. Register coverage and diagnostic practice can differ between those periods. Check that the birth cohorts you compare fall inside a period of stable coverage in your country.

6. Step 4. Handle a second trait (optional)

Skip this step unless probands with another diagnosis should be removed or stop being at risk. To use it, pass `secondaryPhenotype` and `trait2`, then make three choices.

Choice 1. Which side of the index time. `exclude_trait2` is `"before"`, `"after"` or `"any"`. An onset at exactly the index age counts as before.

Choice 2. How far away. `trait2_window` is a gap in years. `trait2_window_type = "within"` flags gaps up to the window. `"at_least"` flags gaps of the window or more. Leave `trait2_window` empty to flag the whole side.



second-trait windows · 4 settings

The bars that reach the edge of the axis continue beyond it.

Choice 3. What happens to a flagged proband.

`trait2_action`	Effect	Allowed with
`"exclude"` (default)	Removed completely, including all their years at risk	before, after, any
`"competing"`	Kept, recoded to Status 2 with `FailureTime` set to the age the second trait started	before only

The function's own examples call `"competing"` the preferred alternative for a second trait that comes first. The proband then contributes time at risk up to the second trait and leaves as a competing event.

Three consequences to know. The flag applies to every proband, not only to cases, and for a non-case the index `FailureTime` is the censoring age. With `"competing"`, an index case whose second trait came first is no longer a case in the output. With `max_followup` set, the comparison uses the proband's time after the window was applied, and second-trait onsets after the window are ignored.

7. Step 5. Run the function and read the log

Make one call per kind of relative. Object names below (`main`, `genealogy`, `bip`, `dep`) stand for your own tables.

Basic call, full siblings

This is the smallest call that works. Nobody is removed or censored, and siblings are counted at any age.

```
fs <- make_epimight_tte(mainPhenotype = main, genealogy = genealogy,
                        trait = "MDD", relatives_kind = "FS",
                        relatives_label = "full_siblings")
```

Basic call, parents

```
par <- make_epimight_tte(mainPhenotype = main, genealogy = genealogy,
                         trait = "MDD", relatives_kind = "PO",
                         relatives_label = "parents",
                         vertical = "older", max_relatives = 2)
```

Fixed observation time, probands for 20 years and parents for 50

```
par20 <- make_epimight_tte(mainPhenotype = main, genealogy = genealogy,
                           trait = "MDD", relatives_kind = "PO",
                           relatives_label = "parents",
                           vertical = "older", max_relatives = 2,
                           max_followup = 20,
                           max_followup_relatives = 50,
                           study_end = "2016-12-31")
```

What the three extra arguments do in this call.

- `max_followup = 20` gives every proband exactly 20 years of observation from birth. Probands born after 1996 are removed, because they cannot be observed for 20 years before the data end. A proband diagnosed at 25 becomes Status 0 at age 20. A proband diagnosed at 12 is unchanged.
- `max_followup_relatives = 50` does the same for the parents with 50 years. A parent diagnosed at 55 still counts as a relative but not as an affected one. A parent born after 1966 is not counted at all.
- `study_end` is the date the data end, which both windows are measured against.

If you leave out `max_followup_relatives`, the parents get the same 20 years as the probands. Set it to `NULL` to count relatives at any age. Section 5 explains the rules in full.

Minimum follow-up only

```
fs <- make_epimight_tte(mainPhenotype = main, genealogy = genealogy,
                        trait = "MDD", relatives_kind = "FS",
                        relatives_label = "full_siblings",
                        min_followup = 10, study_end = "2016-12-31")
```

BIP before MDD ends MDD risk as a competing event

```
fs <- make_epimight_tte(mainPhenotype = main, genealogy = genealogy,
                        trait = "MDD", relatives_kind = "FS",
                        relatives_label = "full_siblings", min_followup = 10,
                        secondaryPhenotype = bip, trait2 = "BIP",
                        exclude_trait2 = "before", trait2_action =
"competing")
```

Exclude probands who had BIP before, or at the same age as, MDD

```
fs <- make_epimight_tte(mainPhenotype = main, genealogy = genealogy,
                        trait = "MDD", relatives_kind = "FS",
                        relatives_label = "full_siblings", min_followup = 10,
                        secondaryPhenotype = bip, trait2 = "BIP",
                        exclude_trait2 = "before")
```

Possible misdiagnosis, BIP probands who get MDD within 2 years after

```
bip_fs <- make_epimight_tte(mainPhenotype = main, genealogy = genealogy,
                            trait = "BIP", relatives_kind = "FS",
                            relatives_label = "full_siblings",
                            secondaryPhenotype = dep, trait2 = "MDD",
                            exclude_trait2 = "after", trait2_window = 2)
```

Reading the log

Each run prints its messages in this order. N and M stand for counts from your data. A line appears only when its argument is set.

Order	Message printed	Appears when	What to check
1	N Individual(s) removed due to negative TTE or NA	Always	N is small and explainable
2	End of follow-up derived from data, followed by a date	A follow-up argument is set and `study_end` is empty	The date equals your administrative end date
3	N of M proband(s) removed because they could not be observed for X years	`max_followup` set	The "born after" date is the cutoff you intended
4	N of M proband(s) censored at X years, split into diagnosed later, competing event later and censored later	`max_followup` set	"Diagnosed later" is the number of cases you give up
5	N of M proband(s) removed with less than X years of potential follow-up	`min_followup` set	The "born after" date is the cutoff you intended
6	N of M proband(s) with the second trait, split into cases and non-cases, then "excluded" or "recoded as competing event"	`exclude_trait2` used	The split between cases and non-cases is plausible
7	N of M relative(s) removed, then N of M relative(s) censored	`max_followup_relatives` set	Same two checks as lines 3 and 4, for relatives
8	relatives_n distribution, followed by a frequency table	Always	The largest value is possible for the kind, for example 2 for parents

The frequency table covers only people with at least one relative. People with none are added later with a count of 0.

8. Step 6. Check the output

Each call returns a `data.table` sorted by `person_id`, with one row per proband who passed the filters.

Column	Type	Content
`person_id`	character	`ID` from `mainPhenotype`
`trait`	character	The `trait` argument
`trait_status`	integer	0, 1 or 2, after any censoring by `max_followup` and any recoding by `trait2_action = "competing"`
`trait_age`	integer	`floor(FailureTime)`, completed years of age, never above `max_followup`
`relatives_kind`	character	The `relatives_label` argument, the text run_h2 selects on
`relatives_n`	integer	Relatives of this kind with phenotype data, 0 if none
`relatives_n_trait`	integer	Of those, the number with Status 1 inside their window
`birth_year`	integer	Year of `BirthDate`, kept by default
`sex`	character	`Sex` from `mainPhenotype`, kept by default

```
person_id,trait,trait_status,trait_age,relatives_kind,relatives_n,relatives_n
_trait,birth_year,sex
1001,MDD,1,24,full_siblings,2,0,1980,F
1002,MDD,0,39,full_siblings,2,1,1982,M
1005,MDD,0,33,full_siblings,0,0,1988,F
```

Two attributes ride on the table. Save the relatedness to a variable straight away, because the pooled table should not be relied on to keep it.

```
attr(fs, "kinship")       # median kinship of the pairs with the chosen codes
r_fs <- attr(fs, "relatedness")   # 2 x kinship, the value for run_h2
```

Checklist before you go on

- [ ] The count of rows removed for negative or missing time is small and explained.
- [ ] The end of follow-up equals your administrative end date.
- [ ] With a window, no `trait_age` is above `max_followup`, and the youngest `birth_year` matches the cutoff in the log.
- [ ] The `relatives_n` table looks right for the kind. Parents never exceed 2.
- [ ] There is no warning about several kinship values, or you know why the pooled codes differ.
- [ ] The relatedness matches the expected value for the kind (0.5 for full siblings and parents).
- [ ] The share of probands with `relatives_n = 0` is plausible for your register coverage and your relatives window.
- [ ] Cases in the output (`trait_status == 1`) match the number you expect after the filters.

9. Step 7. Pool the tables and run epimight

Stack the tables from all kinds into one pool, then give the pool to epimight.

```
pool <- rbind(fs, par)   # one row per person x trait x relatives_kind

pipeline <- epimight::Pipeline$new(pool = pool)
pipeline$run_h2(
  cif_pop = list(index_trait = "MDD", stratify_columns = list("birth_year")),
  cif_fh  = list(index_trait = "MDD", relatives_trait = "MDD",
                 relatives_kind = "full_siblings",
                 stratify_columns = list("birth_year")),
  relatedness = r_fs
)
```

Three rules keep the pool consistent.

1. The `relatives_kind` given to run_h2 equals the `relatives_label` you used in the call.

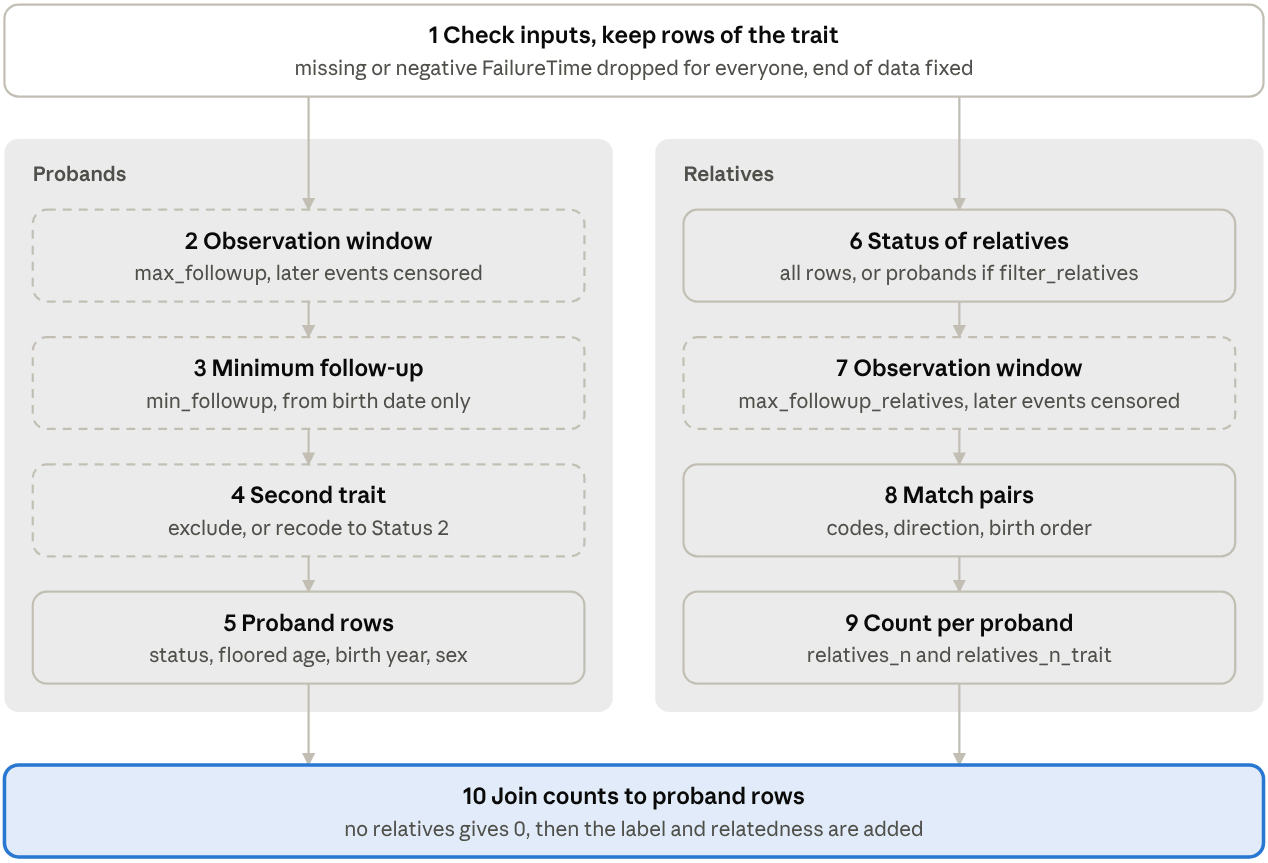
2. The `relatedness` belongs to that same kind.
3. Every kind in one pool was built with the same trait and the same proband settings (`max_followup`, `min_followup`, `study_end` and the second-trait arguments). Otherwise the same person enters the population CIF with a different status or age across kinds.

`max_followup_relatives` may differ between kinds, because it changes the relatives' counts and not the proband rows. The pool should have as many rows as probands times kinds.

10. How the function works inside

The function prepares the probands and the relatives separately and joins them at the end. The numbers give the order in which the code runs.

Ten steps inside the function, probands on the left and relatives on the right



Dashed boxes are optional and run only when their argument is set.

inside the function · 10 steps, 2 lanes

Three points follow from this order. The proband window runs before the minimum follow-up filter and before the second-trait step. The relatives get their own window, independent of the proband steps, unless `filter_relatives = TRUE` makes step 6 start from the filtered probands. The join in step 10 starts from the proband rows, so a person who appears only as a relative never gets an output row.

11. Argument reference

Input data

Argument	Default	What it does
`mainPhenotype`	required	Index phenotype table, used for probands and for relatives' status
`genealogy`	required	Table of relative pairs
`secondaryPhenotype`	`NULL`	Second phenotype table, needed when `exclude_trait2` is not `"none"`

What to extract

Argument	Default	What it does
`trait`	required	Index phenotype name, exact match on `Phenotype`
`relatives_kind`	required	One or more relationship codes. Several codes are pooled under one label
`relatives_label`	codes joined with `+`	Text written to the output column `relatives_kind`

Which relatives to count

Argument	Default	What it does
`vertical`	`"none"`	Birth-order filter. `"older"` for parents and grandparents, `"younger"` for children. `TRUE` means older, `FALSE` means none
`relative_is`	`"either"`	Which genealogy column holds the relative. `"id2"` or `"id1"` for directional files
`max_relatives`	`NULL`	Sanity check only. Warns with example IDs when a person has more relatives than this
`filter_relatives`	`FALSE`	`TRUE` takes the relatives from the filtered probands, so proband removals and recodings also apply to relatives

Observation time

Argument	Default	What it does
`max_followup`	`NULL`	Fixed window for probands, in years from birth. Removes probands born too late and censors everything after the window at Status 0
`max_followup_relatives`	value of `max_followup`	The same window for relatives, with its own number of years. `NULL` means no window for relatives
`min_followup`	`NULL`	Minimum potential follow-up for probands in years, from birth date only. Nothing is censored
`study_end`	`NULL`	End of data as a date, used by the three arguments above. Derived from the data and printed if not given

Second trait

Argument	Default	What it does
`trait2`	`NULL`	Second phenotype name
`exclude_trait2`	`"none"`	`"before"`, `"after"` or `"any"`, timing of the second trait against the index `FailureTime`
`trait2_window`	`NULL`	Gap in years between the two events. `NULL` means any gap
`trait2_window_type`	`"within"`	`"within"` flags gaps up to the window. `"at_least"` flags gaps of the window or more
`trait2_action`	`"exclude"`	`"exclude"` removes flagged probands. `"competing"` recodes them to Status 2 at second-trait onset, only with `"before"`

Output

Argument	Default	What it does
`keep_columns`	`c("birth_year", "sex")`	Which of these two extra columns to keep, for `stratify_columns` in epimight

12. Troubleshooting

Errors that stop the run

Message starts with	Cause	Fix
mainPhenotype is missing columns	A required column is absent or spelled differently	Rename to `ID`, `Sex`, `BirthDate`, `Phenotype`, `Status`, `FailureTime`
genealogy is missing columns	Same, for the genealogy	Rename to `id1`, `id2`, `relationship_kind`, `kinship`

Message starts with	Cause	Fix
secondaryPhenotype is missing columns	Same, for the second table	Rename to `ID`, `Phenotype`, `Status`, `FailureTime`
No rows in mainPhenotype for trait	`trait` does not match any value of `Phenotype`	Check spelling and case
Duplicate IDs for trait	A person has more than one row for the trait	Reduce to one row per person per trait
Status must be 0, 1 or 2	Another code or a missing value in `Status`	Recode before calling
BirthDate could not be parsed	Dates are not in a format R reads	Convert to Date or to text like 1985-03-14
max_followup must be one positive number of years	The window is zero, negative, text or more than one value	Give one number, for example 20
max_followup_relatives must be one positive number of years	Same, for the relatives' window	Give one number, or `NULL` for no window
No status 0 individuals to derive the end of follow-up	A follow-up argument is set, `study_end` is empty and nobody is censored	Set `study_end`
trait2 is required, or secondaryPhenotype is required	`exclude_trait2` is used without them	Pass both
trait2_action = 'competing' only works with exclude_trait2 = 'before'	Competing recode asked for `"after"` or `"any"`	Use `"before"`, or use `"exclude"`

Message starts with	Cause	Fix
trait2_window must be a non-negative number	Window is negative or not numeric	Give years as a number
No pairs in genealogy with relationship_kind in	None of the codes exist in your file	Check the codes with `table(genealogy$relationship_kind)`
No probands left after filtering	The filters removed everyone	Check `study_end`, `max_followup` and `min_followup` first
unused argument	An argument name is misspelled	Compare with section 11

Warnings that let the run finish

Warning	Meaning	What to do
No cases (Status == 1) of trait2 found	Nobody has the second trait, so nobody is flagged	Check the spelling of `trait2` and the Status coding
N person(s) have more than K relatives	Someone exceeds `max_relatives`	Look at the example IDs, then check `relative_is`, `vertical` and the codes
Multiple kinship values, using the median	The chosen codes carry different kinship values	Do not pool kinds with different relatedness. Check the genealogy for the values listed

Output that looks wrong without any message

Symptom	Likely cause
Almost everyone has `relatives_n = 0`	ID formats differ between the phenotype and genealogy tables, unaffected people have no phenotype row, or the relatives' window removed relatives born too late
Almost no relative is affected	The relatives' window is shorter than the usual age at onset. Remember that it defaults to `max_followup`

Symptom	Likely cause
Parents have more than 2 relatives	A symmetric `PO` code was read both ways without `vertical = "older"`
A stratifier column is missing from the output	`keep_columns` only returns `birth_year` and `sex`
Fewer cases than expected	`max_followup` censored later diagnoses, `min_followup` removed recent birth years, or `"competing"` recoded cases to Status 2
Recent birth years are missing	`max_followup` or `min_followup` removed people born too late for the chosen number of years

13. Known limitations

The script has been tested on real Danish register data that end in 2016. The points below come from reading the code as it stood on 6 October 2026.

Behaviour	Consequence	What to do for now
The observation window always starts at birth	Only onset up to the window age is studied. A window that starts later in life is not possible	Choose the window with the age at onset of the trait in mind
The window fixes the length of observation, not the calendar period	Cohorts can still differ in register coverage and diagnostic practice	Compare cohorts inside one period of stable coverage
`max_followup_relatives` defaults to `max_followup`	A short proband window silently applies to parents too	Set the relatives' window explicitly for older kinds of relative
The relatives' window removes relatives born too late	Counts for younger kinds, such as children, can become very small	Check the printed `relatives_n` table for each kind

Behaviour	Consequence	What to do for now
`filter_relatives = TRUE` combined with two different windows	Relatives start from the windowed probands, so the shorter window wins	Keep `filter_relatives = FALSE` when the windows differ
`keep_columns` can only select `birth_year` and `sex`	Any other column in `mainPhenotype` is not carried to the output, with no warning	Merge extra stratifiers onto the output by `person_id`
The kinship attribute is taken from all genealogy pairs with the chosen codes	It is computed before the phenotype, window and birth-order filters, so it is not strictly the pairs used	Fine when a code has one kinship value. Check the warning otherwise
`vertical` compares birth dates with a strict inequality	Relatives born on the same day are dropped under `"older"` and `"younger"`	Use `"none"` for sibling kinds
`"either"` on a parent and offspring code counts parents and children together	A parent appears to have their children as relatives of kind PO	Always set `vertical`, or `relative_is` for directional files
`trait2_action = "competing"` recodes every flagged proband	A case whose second trait came at or before the index diagnosis becomes Status 2 at the earlier age	Intended. Report the printed counts of recoded cases and non-cases
`secondaryPhenotype` onset ages are not checked for negative values	With `"competing"` a negative onset would give a negative `trait_age`	Clean the secondary file first
`trait_age` is floored	A diagnosis in the first year of life gives age 0	Confirm epimight accepts age 0 for your trait
The derived end of follow-up uses Status 0 rows of this trait only	Wrong if nobody is censored at the administrative end	Set `study_end` explicitly per country

Behaviour	Consequence	What to do for now
A column literally named `trait` or `trait2` in an input table	The trait filter would compare against that column, not the argument	Rename such columns before calling
The usage comment at the end of the code file pools `rbind(fs, po)`	The object is called `par`, and that example uses BIP while the pool is for MDD	Use the calls in section 7 of this manual